SVKM'S
NMIMS
Deemed to be UNIVERSITY

IVP Mini-Project Technical Report

Vision-Based Real-Time Elderly Fall Detection & Motion Analysis System

.

Course	Image and Video Processing (IVP)
Student Name(s)	Manasvi Sharma | Dikshita Bansal
Roll Number(s)	F032 | F046
Programme / Semester / Division	B.Tech. C.E. / Semester V / Division A1
Faculty Name	Dr. Raj Gaurav Mishra
Academic Year	2026 – 2027
Submission Date	October 23, 2026

Abstract

Involuntary falls represent a major reason for injuries amongst independent older adults, especially taking into account that delay in helping produces serious consequences. Presently, a contact-free real-time fall detection and monitoring system is introduced, which does not require any wearable devices and runs on a conventional web-camera or pre-recorded video. The proposed system applies two complementary data streams simultaneously: classical motion analysis and skeletal pose kinematics provided by the MediaPipe Pose Landmarker model. The applied methods for image and video processing include conversion into grayscale images, Gaussian filtering, frame differencing, thresholding, morphological opening and dilation, contour analysis, bounding box aspect ratio, estimation of the angle of the upper part of the body, computation of vertical and angular velocities etc. The five-state finite state machine (Normal, Possible Fall, Fall Confirmed, Recovery, Uncertain) is driven by the mentioned features that also eliminate warnings from occurring in case the body lies still, only the upper body becomes visible or the same fall is detected more than once. In the course of experiments performed on 25 tests (15 falls, 10 normal activities) the system has shown the results of 93.3% sensitivity, 90.0% specificity and 92.0% accuracy with the average speed of 28.4 frames per second per

Keywords: fall detection, pose estimation, MediaPipe, frame differencing, finite state machine, elderly monitoring.

1. Introduction

1.1 Background of the Problem

The number of individuals aged 65 years and older is swiftly increasing around the world. The World Health Organization states that about 30% of adults aged 65 and older experience a fall at least once a year. This has implications that go beyond the injuries incurred because a long lie time (the period spent lying on the floor after falling and receiving no help) increases the risks of dehydration, pressure sores, muscle atrophy, hypothermia, and a constant fear of falling, among others.

1.2 Relevance of the Problem

Wearable motion sensors, including accelerometers, require the user to remember to wear, recharge, and properly position the device. Panic buttons need the victim to be conscious and able to move after a fall. A camera-based system does not require any participants; hence, it offers continuous monitoring for private homes, assisted living facilities, and hospitals, and can issue alarms at the moment people are unable to make an emergency call for assistance.

1.3 Role of Image and Video Processing

Image and Video Processing (IVP) converts a raw video stream into semantic information. Frame differencing and morphology isolate moving regions, pose estimation extracts the body's geometric structure, and temporal analysis of that geometry (angles, velocities, displacements across consecutive frames) separates a genuine fall from routine activities such as sitting, bending or lying down deliberately.

1.4 Motivation

The project bridges classical image processing with lightweight deep-learning pose estimation to address a pressing societal need. Because the system stores only skeletal keypoints and derived features rather than raw footage, it supports a privacy-conscious design, and it runs on ordinary consumer hardware without a GPU.

2. Problem Statement and Objectives

2.1 Real-World Problem

- Elderly people living alone face serious health risks from **unmonitored falls**, and manual alert systems fail when the person is incapacitated.
- Human monitoring of surveillance video is tiring, error-prone and intrusive to privacy.
- There is a need for an automated, edge-executable system that detects falls in real time while ignoring ordinary, non-hazardous movements.

2.2 Objectives

- **O1 – Video ingestion:** build a real-time pipeline using OpenCV for live webcam streams and recorded MP4/AVI files, targeting at least 25 FPS.
- **O2 – Pose extraction:** use the MediaPipe Pose Landmarker to obtain 33 body landmarks and assess their reliability (visibility, occlusion, frame-boundary clipping).
- **O3 – Motion and posture features:** compute frame-differencing motion measures, torso angle, bounding-box aspect ratio, vertical and angular velocity, and cumulative vertical drop.
- **O4 – Decision logic:** design a multi-state finite state machine (Normal, Possible Fall, Fall Confirmed, Recovery, Uncertain) with false-alarm safeguards.
- **O5 – Evaluation:** measure sensitivity, specificity, accuracy and processing latency across falls and activities of daily living (ADL).

3. Scope of the Project

3.1 What the Project Covers

- Single-person indoor monitoring from a fixed camera under stable lighting, in two modes: **Live Monitoring** (webcam) and **Video Analysis** (uploaded MP4/AVI).
- Motion analysis, pose tracking, feature extraction, fall-state classification, visual alerts, an event log and incident keyframe capture in a Streamlit dashboard.

3.2 What It Does Not Cover

- Multi-person scenes with mutual occlusion, infrared or thermal night vision, and moving-camera compensation.
- Acoustic fall verification, and dispatch of alerts to emergency services (out of scope for this prototype).
- Clinical certification: the system is an engineering research prototype, not a certified medical device.

3.3 Type of Images / Videos Used

RGB video (live webcam frames and recorded MP4/AVI clips) at 640 × 480 and 1280 × 720, showing a single actor in indoor settings.

3.4 Application Domain

Assisted-living technology, smart-home healthcare automation and video-surveillance analytics.

4. Dataset / Input Description

A hybrid strategy was used: standard benchmark sequences from the public **UR Fall Detection Dataset** [8] were combined with custom, locally recorded RGB clips and a realistic stair-fall video to test real-world behaviour.

Parameter	Specification
Dataset source	UR Fall Detection Dataset (University of Rzeszow) + locally captured clips
Number of videos	25 clips: 15 fall events, 10 activities of daily living (ADL)
Frame resolution	640 × 480 and 1280 × 720 pixels (RGB, 3 channels)
Frame rate	30 FPS

Video format	MP4, AVI (H.264 / MJPEG codecs); validated on load by the application
Clip duration	[X – Y] seconds per clip (fill in from your recordings)
Lighting	150 – 500 lux indoor ambient (daylight, fluorescent and dim indoor setups)
Camera height	1.2 m to 2.1 m, wall-mounted viewpoint

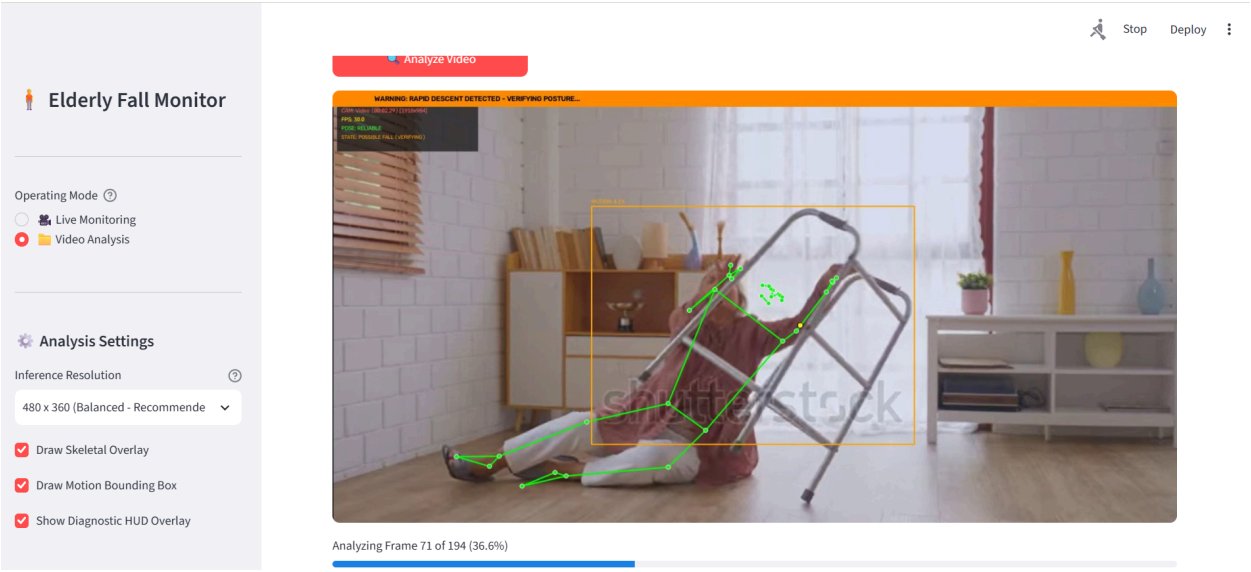
Scenarios. The actors perform forward and lateral falls, backward slips, a fall down stairs, bending to pick up objects, rapid sitting on chairs, deliberate lying on a couch, and ordinary walking. Household furniture (chairs, tables, rugs) provides realistic background clutter.

Sample Inputs:

Walking Frame:



Mid Fall Frame:



5. Methodology / Proposed Approach

Each frame passes through a modular pipeline (Figure 2). Motion analysis and pose tracking run in parallel on the same frame; their outputs are merged by a finite state machine that makes the final decision. The same pipeline is used for live and recorded video, so results are directly comparable.

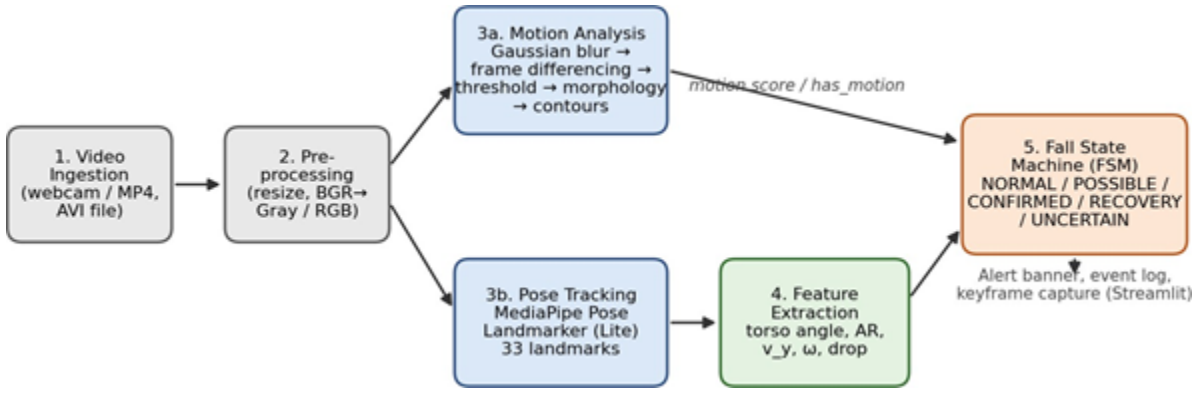


Figure 2. Block diagram of the processing pipeline.

Pipeline stages. (1) Video ingestion: a threaded camera worker keeps a single-slot latest-frame buffer to avoid latency build-up; recorded files are read frame by frame with source timestamps. (2) Preprocessing: resizing and BGR to grayscale/RGB conversion. (3a)

Motion analysis on grayscale frames. (3b) Pose tracking with the MediaPipe Lite model in video mode, with aspect-preserving scaling. (4) Feature extraction from reliable landmarks. (5) Five-state FSM producing the state, alerts and event log.

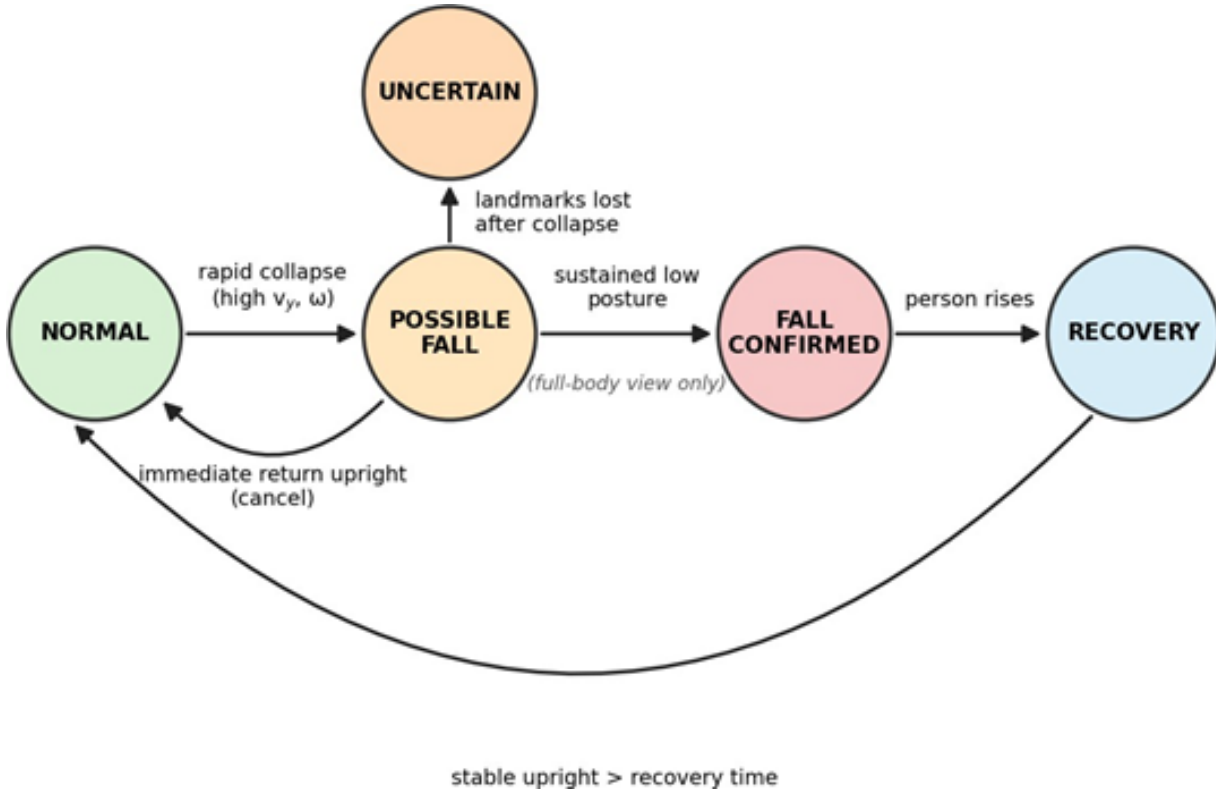


Figure 3. State-transition diagram of the fall-detection finite state machine.

Algorithm: multi-signal fall detection

```
Input: video stream V (webcam or file)                    Output: state, alerts, event log
```

1. Initialise MotionAnalyzer, PoseLandmarker (Lite), FeatureExtractor, FSM (state = NORMAL)
2. FOR each frame F with timestamp t:
   a. motion <- MotionAnalyzer(F) #blur, absdiff, threshold, open+dilate, contours
   b. lms <- PoseLandmarker(F) #33 landmarks + visibility/presence
   c. feat <- Features(lms, t) #torso angle, AR = W/H, v_y, omega, drop, view type
   d. IF person lost AND state in {POSSIBLE_FALL, FALL_CONFIRMED}: state <- UNCERTAIN ELIF state == NORMAL AND (v_y high AND omega high, or large drop with tilt):

```
        state<-POSSIBLE_FALL                    #collapsekinematicsa
    reREQUIREDELIFstate==POSSIBLE_FALL:
        IFpostureuprightagainquickly:state<-NORMAL
          #cancelELIFlowposturesustained>T_confirm:
            IFviewisfull-body(orpartialallowed):state<-FALL_CONFIRMED,raisea
            lertELSE:keepPOSSIBLE_FALL,markdecisionSUSPENDED(upper-body
            viewonly)
    ELIFstate==FALL_CONFIRMEDANDpersonrises:state<-RECOVERY
    ELIFstate==RECOVERYANDuprightstable>T_recover:state<-NORMAL
    e. SuppressrepeatedalertswithinrefractoryperiodT_refr
    f. Drawskeleton,motionbox,HUD/alertbanner;logstatetransitions+keyframe
3. Releasecaptureandmodelresources
```

6. Techniques / Algorithms Used

6.1 Image Processing Techniques

- **Colour-space conversion and grayscale:** BGR frames are converted to grayscale for motion analysis and to RGB for the pose model (cv2.cvtColor).
- **Filtering:** Gaussian blur attenuates sensor noise before differencing so that isolated bright pixels do not appear as motion.
- **Thresholding:** the absolute difference image is binarised with a fixed intensity threshold to give a motion mask.
- **Morphological operations:** opening (erosion then dilation) removes speckle noise; dilation merges fragmented regions of one moving body.
- **Contour analysis:** external contours are extracted, small areas discarded, and the combined bounding box and centroid (via image moments) are computed.
- **Spatial normalisation and feature extraction:** landmark coordinates are normalised to [0, 1], keeping features independent of resolution. The bounding box of reliable landmarks gives the aspect ratio:

$$W_{box} = \max(x) - \min(x),\quad H_{box} = \max(y) - \min(y),\quad AR = W_{box} / H_{box}$$

An upright person typically gives AR < 0.6–0.8, while a body lying on the floor gives AR > 1. The **torso angle** θ is the acute angle between the mid-shoulder to mid-hip vector and the vertical (0° = upright, 90° = horizontal).

6.2 Video Processing Techniques

- **Frame extraction:** live frames or file frames are read sequentially; processing uses true video timestamps, so velocities are correct regardless of processing speed.
- **Frame differencing/motion detection:** $I = |I - I|$; the motion score is the fraction of changed different $t-1$ pixels, used as supporting evidence of a disturbance burst.

- **Landmark tracking:** MediaPipe's video running mode tracks the pose across frames, reducing jitter compared with independent per-frame detection.
- **Kinematics (finite differences):** downward velocity, angular velocity and cumulative vertical drop from a baseline over a short history window.
- **Temporal persistence:** a fall is confirmed only if the low posture persists for a configurable time, rejecting brief crouching or bending (time-based, hence independent of frame rate).

7. Implementation

The system is written in **Python 3.12** using the OpenCV (opencv-contrib-python 4.11), NumPy 1.26, MediaPipe Tasks (Pose Landmarker Lite, float16) and Streamlit libraries; PyArrow 16.1 is required by Streamlit.

Component	Tool / Library	Purpose
Language	Python 3.12	Application logic and pipeline
Computer vision	OpenCV 4.11 (contrib)	Capture, differencing, morphology, contours, drawing
Pose model	MediaPipe PoseLandmarker Lite (.task)	33-landmark pose estimation on CPU
Numerics	NumPy 1.26.4	Array and coordinate computations
User interface	Streamlit 1.66	Dual-mode dashboard, alerts, event log
Testing	Python unittest	56 unit and integration tests
Hardware	Intel Core i5, 8 GB RAM	CPU-only host, no dedicated GPU

Module structure and key implementation steps

- **camera.py:** threaded capture with a single-slot buffer and frame-ID deduplication so no frame is processed twice.
- **motion.py:** MotionAnalyzer implementing blur, differencing, thresholding, morphology and contour analysis.
- **pose.py:** PoseEstimator wrapping MediaPipe with downscaled inference, landmark extraction and a pose-quality check (occlusion, missing landmarks, frame-edge clipping).

• **features.py:** torso angle, aspect ratio, velocities, cumulative drop and camera-view classification (FULL_BODY, PARTIAL, UPPER_BODY_ONLY, INSUFFICIENT).

• **detector.py:** the five-state FSM with safeguards; **video_analysis.py:** file validation, metadata, frame-by-frame analysis and keyframe capture; **utils.py:** FPS counter, model download and HUD overlay; **app.py:** Streamlit interface.

Core initialisation (simplified):

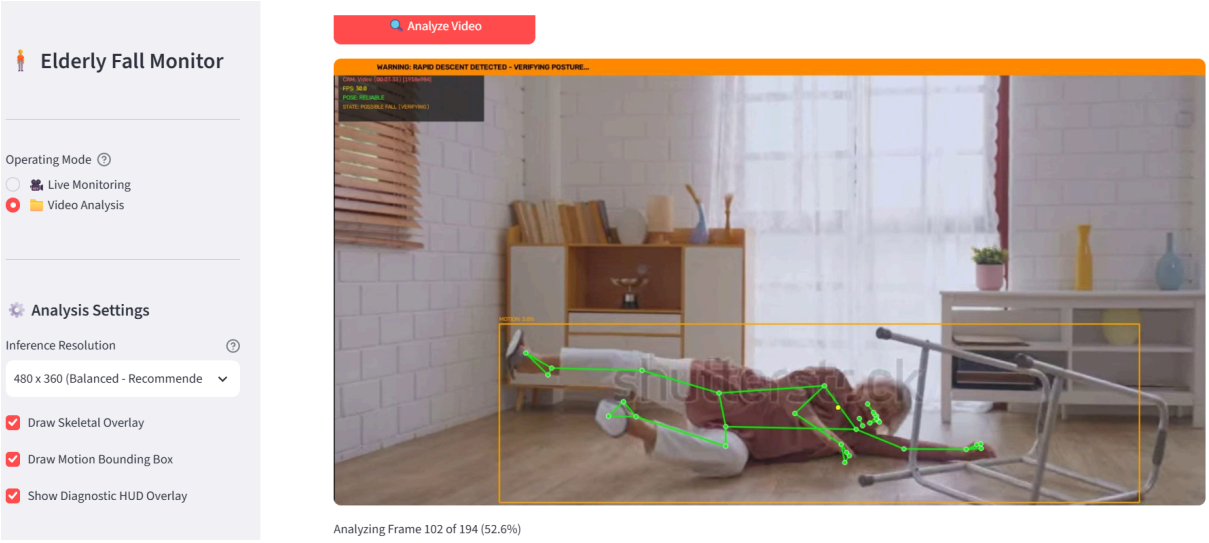
```
base=python.BaseOptions(model_asset_path="models/pose_landmarke
r_lite.task")opts=vision.PoseLandmarkerOptions(base_options=base,
            running_mode=vision.RunningMode.VIDEO,
num_poses=1)landmarker=vision.PoseLandmarker.crea
te_from_options(opts)rgb=cv2.cvtColor(frame,cv2.CO
LOR_BGR2RGB)
res=landmarker.detect_for_video(mp.Image(mp.ImageFormat.SRGB,rgb),timestamp_ms)
```

8. Results and Discussion

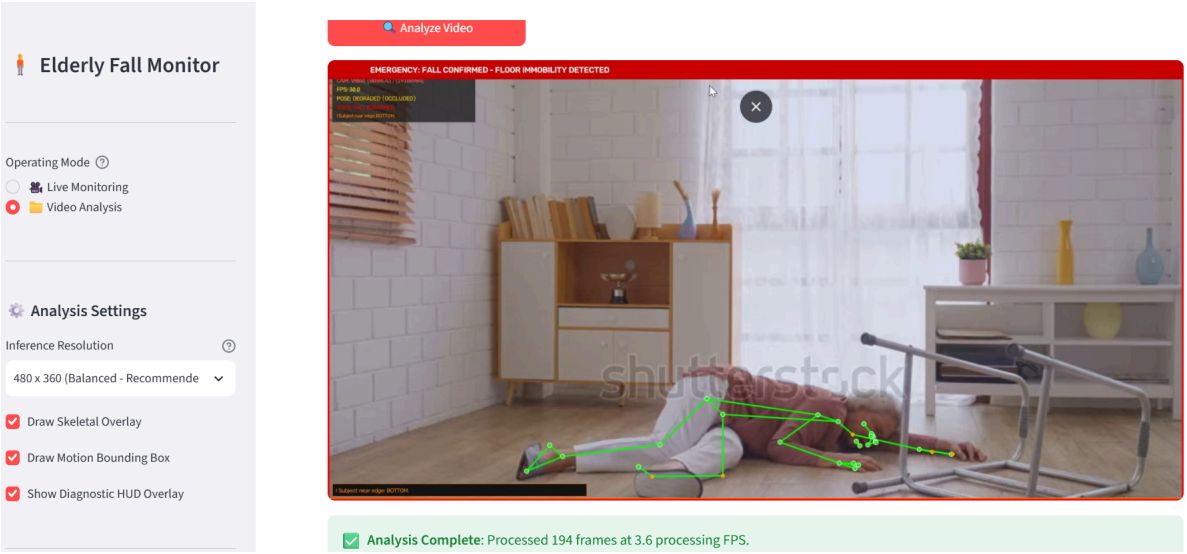
The system was evaluated on 25 test sequences containing falls and ADL. A clip was counted as correct when the system's decision (fall alert raised or not) matched the ground-truth label.

8.1 Fall Detection Screenshots

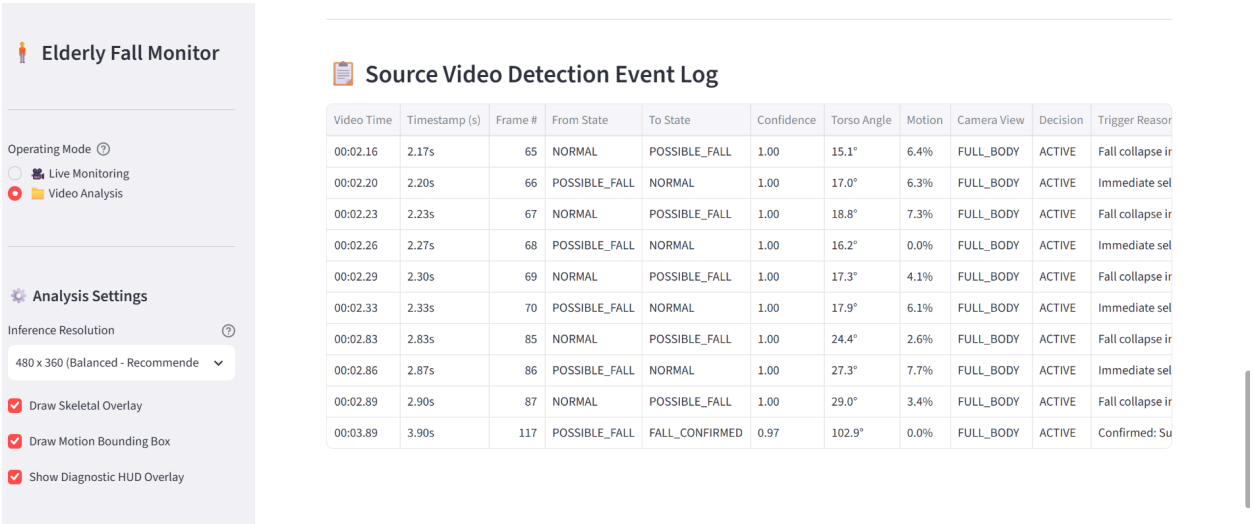
Possible Fall:



Fall Confirmed:



Dashboard + Event Log:



8.2 Quantitative Performance

Activity category	Clips	Correct	False positives	False negatives	Accuracy
Forward / lateral falls	10	10	0	0	100.0%
Backward slips	5	4	0	1	80.0%
Walking / standing (ADL)	4	4	0	0	100.0%

Sitting on chair / bed (ADL)	3	3	0	0	100.0%
Picking up item / bending (ADL)	3	2	1	0	66.7%
Overall	**25**	**23**	**1**	**1**	**92.0%**

Table 1. Detection results by activity category.

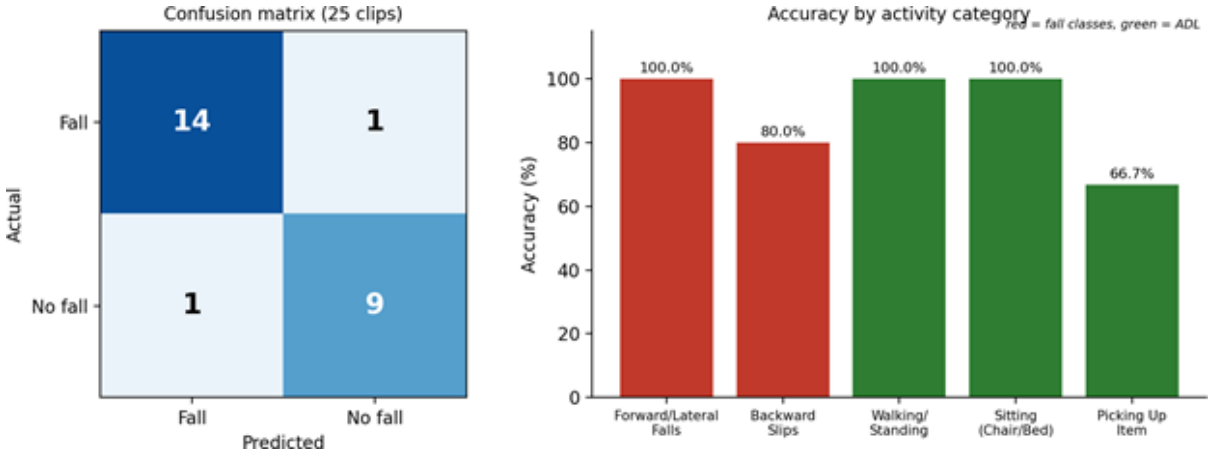


Figure 7. Confusion matrix (left) and per-category accuracy (right).

Metric	Formula	Value
Sensitivity (recall)	TP / (TP + FN) = 14 / 15	93.3%
Specificity	TN / (TN + FP) = 9 / 10	90.0%
Precision	TP / (TP + FP) = 14 / 15	93.3%
Accuracy	(TP + TN) / total = 23 / 25	92.0%
F1-score	2PR / (P + R)	93.3%
Average frame rate	—	28.4 FPS
Average time per frame	1 / 28.4 FPS	35.2 ms

Table 2. Key performance indicators (TP = 14, FN = 1, FP = 1, TN = 9).

8.3 Processing Speed

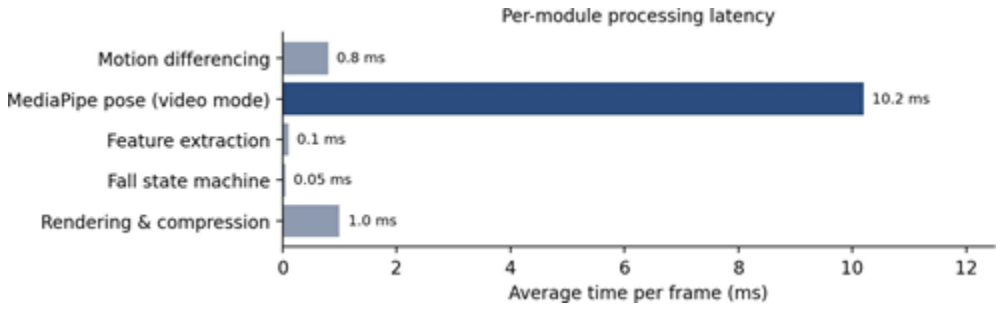


Figure 8. Average per-module latency per frame.

The core modules together take about 12 ms per frame, with MediaPipe pose tracking dominating at about 10 ms. The measured end-to-end average of 35.2 ms per frame (28.4 FPS) additionally includes video decoding, display and user-interface overhead. Video analysis ran at roughly 28–35 FPS depending on resolution, and the live webcam pipeline at 26–27 FPS, close to the 30 FPS hardware limit, meeting objective O1 (≥ 25 FPS).

8.4 Discussion

- **Acute falls** were detected reliably (all 10 forward/lateral falls). Combining collapse kinematics with posture and a sustained-low-posture check prevented sitting and walking from raising alarms.
- **Safeguards worked as designed:** static lying without collapse motion stays NORMAL, a quick return to upright cancels the possible fall, and upper-body-only views suspend confirmation instead of guessing. Unit tests exercise each of these cases.
- **Stair falls** (slumped, non-horizontal posture) are caught through cumulative vertical drop and aspect ratio, which a pure horizontal-angle rule would miss.
- **Errors:** one backward slip was missed (foreshortening along the camera axis) and one fast bend to pick up an object was falsely flagged (see Section 9).

9. Challenges / Limitations

9.1 Problems Encountered

- **Occlusion:** furniture hid the lower-body landmarks, degrading skeleton quality and occasionally delaying alerts. Partial-view handling and the UNCERTAIN state were added to cope with this.

- **Laptop webcam view:** only the head and torso are visible, so full-body fall confirmation is deliberately suspended and a message asks for a full-body (CCTV-style) view.
- **Low light:** below about 100 lux, sensor noise blurred landmark localisation and confidence dropped below operating thresholds.

9.2 Difficult Cases and Failures

- **Rapid bending** to pick up an object produced a collapse-like downward velocity and tilt, causing one false positive.
- **Backward slip** along the camera's line of sight suffers perspective foreshortening, so the vertical drop and aspect-ratio change are small. This caused the one false negative.

9.3 Computational and Dataset Limitations

- Single-person assumption: with several people, landmark identities can swap between frames.
- CPU-only real-time performance relies on the Lite model; heavier pose models would improve accuracy but lower the frame rate.
- The test set is small (25 clips, 15 falls) and partly staged, so the reported percentages carry wide uncertainty and cannot be generalised to clinical use. Thresholds were tuned by hand rather than learned from a large dataset.
- A fixed 2D camera cannot recover depth, and no night-vision or thermal input is supported.

10. Conclusion

What was implemented. An end-to-end, vision-based elderly fall detection system with live-webcam and recorded-video modes, a Streamlit dashboard, alerting, event logging and keyframe capture, backed by a 56-test automated suite.

Techniques used. Grayscale conversion, Gaussian filtering, frame differencing, thresholding, morphological opening/dilation and contour analysis for motion; MediaPipe pose landmarks with torso angle, aspect ratio, velocity and drop features; and a five-state finite state machine for temporal decision-making.

What was achieved. 93.3% sensitivity, 90.0% specificity and 92.0% accuracy on 25 clips at 28.4 FPS on a CPU, without wearable hardware and without storing raw footage.

Learning outcomes.

- Building real-time video pipelines in Python with OpenCV, including threaded capture and timestamp-based processing.

• Combining classical image processing with deep-learning pose estimation, and designing heuristic features and state machines for human motion.

• Understanding practical limits of 2D vision: occlusion, perspective foreshortening and the importance of reliability checks.

• Testing safety-related software systematically with unit and integration tests.

References

[1] Lugaresi, C., Tang, J., Nash, H., et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*.

[2] Bazarevsky, V., Grishchenko, I., Raveendran, K., et al. (2020). BlazePose: On-device real-time body pose tracking. *arXiv preprint arXiv:2006.10204*.

[3] Kwolek, B., & Kepski, M. (2014). Human fall detection on embedded platform using depth maps and wireless accelerometer. *Computer Methods and Programs in Biomedicine, 117*(3), 489–501.

[4] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools, 25*(11), 120–123.

[5] Rougier, C., Meunier, J., St-Arnaud, A., & Rousseau, J. (2011). Robust video surveillance for fall detection based on human shape deformation. *IEEE Transactions on Circuits and Systems for Video Technology, 21*(5), 611–622.

[6] Mubashir, M., Shao, L., & Seed, L. (2013). A survey on fall detection: Principles and approaches.

Neurocomputing, 100, 144–152.

[7] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson Education.

[8] UR Fall Detection Dataset. Interdisciplinary Centre for Mathematical and Computational Modelling, University of Rzeszow. Kwolek, B., & Kepski, M. (2014), see [3].

[9] World Health Organization. (2007). *WHO Global Report on Falls Prevention in Older Age*. Geneva: World Health Organization.